Mini RISC-V Instruction Decoder

A Complete Project Guide in SystemVerilog

Project Title	Mini RISC-V Instruction Decoder
Course	B.Tech, Electronics and Communication Engineering – Final Year
Team Members	Aryan and Abhimanyu
Design Language	SystemVerilog (IEEE 1800)
Simulation Tools	Icarus Verilog (iverilog, vvp) and GTKWave
Editor	Visual Studio Code (Windows)
Version Control	Git and GitHub – working branch merged into main, chunk by chunk

This document is the single reference for the project from the very first line of code to the final submission. It explains, in full depth and assuming no prior background, what the project is, why it is built the way it is, and exactly how it will be built. The work is broken into eight sequential chunks that alternate between the two team members. Every chunk contains the exact files to create, the exact SystemVerilog code with line-by-line reasoning, worked numerical examples, the exact commands to simulate and verify the result, and the exact Git commands to commit, push, and merge that chunk before the next one begins.

Contents Overview

1. Introduction – what this project is and why it matters
2. Theoretical Background – instruction sets, RISC-V, and the six instruction formats
3. Project Definition and Scope – exactly what will and will not be built
4. Development Environment Setup – every tool and VS Code extension, installed step by step
5. Repository Structure and Git Workflow – folders, branches, and the exact commands to use
6. SystemVerilog Essentials – the small subset of the language this project actually needs
7. Implementation Plan – the eight chunks at a glance
8. Chunk 1 (Aryan) – Environment Verification and Repository Skeleton
9. Chunk 2 (Abhimanyu) – Constants Package and the Field Extractor
10. Chunk 3 (Aryan) – Immediate Generator, Part 1 (I, S and U types)
11. Chunk 4 (Abhimanyu) – Immediate Generator, Part 2 (B and J types) and Instruction Classifier
12. Chunk 5 (Aryan) – The Control Unit
13. Chunk 6 (Abhimanyu) – Top-Level Integration
14. Chunk 7 (Aryan) – Self-Checking Testbench and Waveform Verification
15. Chunk 8 (Abhimanyu) – Documentation and Submission Readiness
16. Reference Tables – opcode map, control signal table, and glossary
17. Viva and Presentation Preparation
18. Future Scope and Extensions
19. Troubleshooting Guide

1 Introduction

1.1 What This Project Is, In One Paragraph

Every processor, from the one inside a calculator to the one inside a laptop, works by repeatedly fetching a small binary number called an instruction and figuring out what that number is asking it to do. The circuit that performs this figuring-out is called an instruction decoder. This project builds exactly that circuit for a simplified version of RISC-V, a modern and freely available processor design (an instruction set architecture, explained in Section 2). The circuit will be written in SystemVerilog, a hardware description language used to describe digital circuits the way a program describes software logic, and it will be simulated on a computer rather than built on physical silicon.

1.2 Why This Is a Good Final-Year Project

- It touches the two subjects a communication/electronics engineer is expected to know from digital design: combinational logic design, and hardware description languages.
- RISC-V is an open standard (nobody owns it or charges a licence fee for it), so every design decision in this guide can be checked against a public specification, and the finished project can genuinely be explained and defended in a viva rather than copied blindly.
- The scope is naturally boundable. A decoder is pure combinational logic – there is no clock, no memory, no pipeline hazards to reason about. This keeps the project small enough for two beginners to finish comfortably in the available time, while still being a real, recognisable building block of a real CPU.
- It divides cleanly into independent pieces (extracting fields, generating immediates, generating control signals) which is exactly what is needed to split work between two people using Git without them constantly editing the same lines of the same file.

1.3 What “Instruction Decoder” Actually Means Here

Concretely, the circuit built in this project is a piece of combinational logic that behaves like a translator. On one side, it accepts a single 32-bit binary pattern – one machine instruction, exactly the way it would sit in a computer's memory. On the other side, it produces a set of clearly labelled, individually meaningful signals: which registers are involved, what number (if any) is encoded directly inside the instruction, and which control wires need to switch on so that the rest of a processor (an ALU, a register file, a memory unit) would know how to act on this instruction. The decoder itself never performs the arithmetic or touches memory; it only figures out what should happen and describes that decision on its output pins. This is precisely the role a decoder plays inside every real processor, including the ones manufactured by Intel, AMD, ARM licensees, and every commercial RISC-V core.

> **NOTE:** Nothing in this project needs a physical FPGA board. The entire design is verified inside a simulator on a laptop, which is the normal and expected way to build and grade a digital design mini-project. If a board is available later, the same code can be reused with almost no changes.

2 Theoretical Background

2.1 Instruction Set Architecture (ISA) – The Contract Between Software and Hardware

An Instruction Set Architecture is the agreed vocabulary that software and hardware both understand. It defines the complete list of operations a processor can perform (add two numbers, load a value from memory, jump to a different part of the program, and so on), and it defines exactly how each of those operations is represented as a pattern of bits. A compiler translates a program written in a language such as C into a long sequence of these bit patterns; the processor's job is to read that sequence, one pattern at a time, and act on it. The ISA is what makes this possible: it is a contract that says “if you see this exact bit pattern, it means this exact operation.”

2.2 RISC – Reduced Instruction Set Computer

RISC-V belongs to a philosophy of processor design called RISC. Two ideas from this philosophy matter directly for this project:

- **Fixed instruction width.** Every instruction is exactly 32 bits wide (in the base configuration used here). This is a deliberate simplification compared with older CISC designs (such as x86) where instructions can be anywhere from 1 to 15 bytes long. A fixed width means the decoder never has to figure out “how long is this instruction” before it can even start decoding it – a huge simplification that is precisely what makes a beginner-friendly decoder project realistic.
- **Regular, sliceable encoding.** RISC-V instructions are designed so that fields which mean the same thing (such as which register is the destination) sit in the same bit positions across as many instruction varieties as possible. This regularity is what allows the decoder to be built mostly out of simple, fixed bit-slicing rather than a huge lookup table.

2.3 Where RISC-V Fits

RISC-V is an open ISA originally developed at the University of California, Berkeley, and now maintained by RISC-V International, a non-profit consortium. Unlike ARM or x86, its specification is publicly published and free to implement without a licence fee, which is why it is used extensively in academic teaching and increasingly in real commercial chips. For this project, the only fact that matters is that its base integer instruction set (called RV32I, meaning 32-bit registers, integer operations only) is small, well documented, and exactly the right size for a mini-project.

2.4 Registers

RV32I defines 32 general-purpose registers, named x0 through x31, each holding a 32-bit value. Two facts about them matter here:

- A register is identified inside an instruction using a 5-bit field, since 2 raised to the power 5 equals 32, exactly enough to name all 32 registers.
- Register x0 is hard-wired to the value zero in a real RISC-V machine; writes to it are discarded and reads from it always return zero. The decoder built in this project does not need to enforce this rule (it only decodes instructions, it does not implement the register file), but it is useful general knowledge and will come up naturally when explaining test cases that use x0.

2.5 The Six Instruction Formats

Every RV32I instruction is 32 bits wide, but those 32 bits are organised differently depending on what the instruction needs to express. There are six such organisations, called formats. Learning these six layouts is the single most important theoretical step in this entire project, because the whole decoder is essentially a circuit that (a) figures out which of these six formats a given instruction uses, and (b) pulls the right bits out accordingly.

2.5.1 R-Type – Register-Register Operations (e.g. ADD, SUB, AND, OR, XOR, SLT)

Used when both operands come from registers and the result also goes into a register.

funct7	rs2	rs1	funct3	rd	opcode
[31:25]	[24:20]	[19:15]	[14:12]	[11:7]	[6:0]

`opcode` says “this is an R-type ALU instruction”, `funct3` and `funct7` together say exactly which operation (add vs subtract vs AND, etc.), `rd` is the destination register, and `rs1`/`rs2` are the two source registers.

2.5.2 I-Type – Immediate and Load Operations (e.g. ADDI, ANDI, ORI, LW)

Used when one operand is a register and the other is a constant number written directly inside the instruction (called an immediate), or when loading a value from memory.

imm[11:0]	rs1	funct3	rd	opcode
[31:20]	[19:15]	[14:12]	[11:7]	[6:0]

The entire 12-bit immediate sits in one contiguous block at the top of the instruction, which makes I-type the simplest format to decode.

2.5.3 S-Type – Store Operations (e.g. SW)

Used to write a register's value out to memory. A store needs to specify two registers (the address base and the value to store) and an address offset, which leaves only two small gaps for the immediate – so the immediate is split into two pieces.

imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
[31:25]	[24:20]	[19:15]	[14:12]	[11:7]	[6:0]

Notice that `rs2`, `rs1`, `funct3` and `opcode` sit in exactly the same bit positions as they do in R-type. This is the “regular encoding” idea from Section 2.2 in action – a decoder can pull out `rs1` the same way regardless of whether the instruction turns out to be R-type or S-type.

2.5.4 B-Type – Conditional Branches (e.g. BEQ, BNE)

Used to jump to a different point in the program only if a condition on two registers is true. Branch targets are always an even number of bytes away, so the hardware never actually stores the least significant bit of the offset – it is always assumed to be zero. This frees up one extra bit of encoding range, so the bits are shuffled into a slightly unusual order shown below.

12	imm[10:5]	rs2	rs1	funct3	imm[4:1]	11	opcode
[31]	[30:25]	[24:20]	[19:15]	[14:12]	[11:8]	[7]	[6:0]

NOTE: The immediate bits are not in numerical order in the instruction (imm[12] comes first, then imm[10:5], then imm[4:1], then imm[11]). This looks strange at first but it is simply the layout that lets `rs1`, `rs2`, `funct3` and `opcode` stay in their usual positions. Section 2.6 explains exactly how to reconstruct the correct numeric value from this scrambled layout. (In the diagram above, the two single-bit columns are labelled just "12" and "11" for space reasons – these mean imm[12] and imm[11] respectively; the row underneath always shows the true instruction bit position.)

2.5.5 U-Type – Upper Immediate (e.g. LUI)

Used to load a large constant into the top 20 bits of a register in one step (the bottom 12 bits are left as zero). This is the simplest format of all – there is no `rs1`, no `rs2`, and no `funct3`.

imm[31:12]	rd	opcode
[31:12]	[11:7]	[6:0]

2.5.6 J-Type – Unconditional Jump (e.g. JAL)

Used to jump to a new location in the program while saving the return address (the address of the next instruction) into `rd`. Like branches, jump targets are always an even number of bytes away, so the same "drop the last bit, shuffle the rest" trick from B-type is used, but over a wider range since jumps can travel much further than branches.

20	imm[10:1]	11	imm[19:12]	rd	opcode
[31]	[30:21]	[20]	[19:12]	[11:7]	[6:0]

2.6 Sign Extension and Immediate Reconstruction

Every immediate value in RV32I can be negative (for example, a branch that jumps backwards, or a load from an offset before the base address), and every one of them is stored as a two's-complement number using fewer than 32 bits. Before that number can be used as a full 32-bit value, it must be sign-extended: the instruction's topmost bit (bit 31, which is always the sign bit of the immediate in every format) is copied into every new bit added on the left. In SystemVerilog this is written using the replication operator, for example `{{20{instr[31]}}, instr[31:20]}` for a 12-bit immediate, which reads as "repeat bit 31 twenty times, then attach the original 12 bits after it", producing a full 32-bit signed number. This single idiom, with the replication count adjusted for how many bits are being extended, is reused for every instruction format in this project.

NOTE: For B-type and J-type, reconstructing the immediate additionally means putting the scrambled bits back into normal numerical order and appending an implicit zero at the bottom, since those immediates always represent an even byte offset. The exact SystemVerilog expressions for every format are given, worked through bit by bit, in Chunks 3 and 4 of this guide (Sections 10 and 11).

2.7 A Complete Worked Example

To make all of the above concrete before any code is written, here is one full instruction decoded entirely by hand. Consider the RISC-V assembly instruction `add x1, x2, x3`, meaning "x1 = x2 + x3". Encoded as a 32-bit R-type instruction, this becomes the hexadecimal value `0x003100B3`. Table 2.1 shows exactly how each field was pulled out of that number.

Field	Bit range	Binary value	Decimal / meaning
opcode	`[6:0]`	`0110011`	R-type ALU operation
rd	`[11:7]`	`00001`	1 (destination is x1)
funct3	`[14:12]`	`000`	selects ADD/SUB group
rs1	`[19:15]`	`00010`	2 (first source is x2)
rs2	`[24:20]`	`00011`	3 (second source is x3)
funct7	`[31:25]`	`0000000`	confirms ADD (not SUB)

Caption – Table 2.1: Manual decode of `0x003100B3` (`add x1, x2, x3`). This exact instruction is used as the first automated test case in Chunk 2 (Section 9), so this hand calculation can be directly compared against the simulator's output later.

3 Project Definition and Scope

3.1 What Will Be Built

A single top-level SystemVerilog module named `riscv_instr_decoder` that accepts one 32-bit instruction and produces, purely combinationally (no clock, no memory of past inputs), every signal a simple single-cycle RISC-V processor's datapath would need in order to execute that instruction. Internally, this top module is built from four smaller, independently testable modules, which is also exactly how the work is split between the two team members:

Sub-module	Responsibility
field_extractor	Slice out opcode, rd, funct3, rs1, rs2 and funct7 from fixed bit positions.
imm_generator	Reconstruct the correct signed 32-bit immediate for whichever format the instruction uses.
instr_type_decoder	Classify the instruction into R / I / S / B / U / J type from its opcode, for readability and debugging.
control_unit	Generate the high-level control signals (reg_write, alu_src, mem_read, mem_write, mem_to_reg, branch, jump, alu_ctrl) that would drive the rest of a processor.

3.2 The Instruction Subset In Scope

Supporting the full RV32I instruction set is not necessary to demonstrate every idea in this project, and would only add repetitive case-statement entries without adding new learning. Instead, this project supports one or two representative instructions per instruction format – enough to touch every format and every category of control signal at least once. Table 3.1 is the authoritative list; nothing outside this table needs to be handled, and the testbenches in every chunk are built around exactly these instructions.

Mnemonic	Format	Opcode	funct3	funct7	Meaning
ADD	R	`0110011`	`000`	`0000000`	rd = rs1 + rs2
SUB	R	`0110011`	`000`	`0100000`	rd = rs1 - rs2
AND	R	`0110011`	`111`	`0000000`	rd = rs1 AND rs2
OR	R	`0110011`	`110`	`0000000`	rd = rs1 OR rs2
XOR	R	`0110011`	`100`	`0000000`	rd = rs1 XOR rs2
SLT	R	`0110011`	`010`	`0000000`	rd = (rs1 < rs2) ? 1 : 0
ADDI	I	`0010011`	`000`	–	rd = rs1 + immediate
ANDI	I	`0010011`	`111`	–	rd = rs1 AND immediate
ORI	I	`0010011`	`110`	–	rd = rs1 OR immediate
LW	I	`0000011`	`010`	–	rd = memory[rs1 + immediate]
SW	S	`0100011`	`010`	–	memory[rs1 + immediate] = rs2

Mnemonic	Format	Opcode	funct3	funct7	Meaning
BEQ	B	`1100011`	`000`	–	if (rs1 == rs2) branch
BNE	B	`1100011`	`001`	–	if (rs1 != rs2) branch
LUI	U	`0110111`	–	–	rd = immediate << 12
JAL	J	`1101111`	–	–	rd = return address; jump

Caption – Table 3.1: The fifteen instructions this decoder must correctly handle. Every one of these appears, encoded to an exact hexadecimal value, in the testbenches built throughout this guide.

3.3 Explicitly Out of Scope

Stating what will not be attempted is just as important as stating what will be, both to keep the project achievable and to be able to answer confidently in a viva when asked “does it support instruction X?”.

- **Execution.** This project decodes instructions; it does not build an ALU, a register file, or a memory, and it never actually computes a sum or performs a memory access. The control signals it produces are exactly the signals such units would need, but wiring up those units is future work (see Section 18).
- **The remaining RV32I instructions**, such as SLTI, SLLI, shifts, AUIPC, JALR, and the unsigned comparison/branch variants. These follow the same formats already covered and can be added later by extending the case statements – no new format or technique would be required.
- **Compressed instructions (the “C” extension)**, which use 16-bit instructions mixed in with 32-bit ones. Supporting these would break the fixed-width assumption this entire mini-project relies on for simplicity.
- **Multiply/divide (the “M” extension), atomics, floating point, or privileged/CSR instructions.** All of these belong to separate, optional RISC-V extensions beyond the base integer set.
- **Exceptions, interrupts and illegal-instruction handling** beyond a simple default case that produces all-zero, harmless control signals for anything not in Table 3.1.

> **NOTE:** If asked in a viva “why doesn't it support every instruction”, the honest and correct answer is that the goal was to demonstrate every instruction format and every category of control signal at least once, not to reproduce the entire RISC-V manual – which is exactly what Section 3.2 and 3.3 together establish.

4 Development Environment Setup

This section only needs to be done once, by each team member, on their own Windows laptop. It installs everything needed to write SystemVerilog in VS Code, simulate it, and look at the resulting waveforms. Nothing here costs money; every tool below is free and open source.

4.1 The Three Tools Needed, and Why

Tool	What it is	Why it is needed
Visual Studio Code	A free source-code editor.	Already chosen by the team; this is where all files will be written and Git will be operated from.
Icarus Verilog (`iverilog` / `vvp`)	A free, open-source Verilog/SystemVerilog simulator that runs from the command line.	Compiles the design and testbench, then runs the simulation and prints the results.
GTKWave	A free waveform viewer.	Displays the value of every signal over simulated time as a graphical timing diagram, so mistakes can be seen visually instead of only read as text.

NOTE: Every command in this guide is a plain Windows command typed into a terminal. No knowledge of Linux is assumed. VS Code's built-in terminal (opened with Ctrl + `, the backtick key) is where all of these commands will be typed.

4.2 Installing Icarus Verilog on Windows

1. Open a web browser and search for “Icarus Verilog Windows download”, or go directly to the project's release page on GitHub (search for `FPGAwars/iverilog` or `steveicarus/iverilog`) and download the latest `.exe` installer for Windows (it is usually named something like `iverilog-v12-...-x64.exe`; any recent version works fine for this project).
2. Run the downloaded installer. Accept the default installation options. On the page that asks about adding Icarus Verilog to the system PATH, make sure that option is ticked – this is important, as it is what allows typing `iverilog` directly into any terminal.
3. Finish the installation, then close and reopen VS Code completely (a full restart, not just a new terminal tab) so that it picks up the updated PATH.
4. Open a new terminal inside VS Code and type `iverilog -V`. If installation succeeded, this prints a version banner such as `Icarus Verilog version 12...`. If instead the terminal says something like `'iverilog' is not recognized`, the PATH was not set correctly – reinstall and make sure the PATH checkbox is ticked, or add the installation folder to PATH manually through Windows' “Edit environment variables” settings dialog.

4.3 Installing GTKWave on Windows

1. GTKWave is bundled with several Icarus Verilog Windows installers; check the terminal command `gtkwave --version` first – if it already prints a version, it is already installed.

2 If it is not found, search for “GTKWave Windows download” and download the standalone installer (commonly distributed via SourceForge or bundled inside toolchains such as the Icarus Verilog installer itself). Run it with default options.

3 Restart VS Code and confirm with `gtkwave --version` in the terminal, the same way as Step 4 for Icarus Verilog above.

4.4 Recommended VS Code Extensions

Extension (search this name in the Extensions panel)	Publisher	What it gives
Verilog-HDL/SystemVerilog	mshr-h	Syntax highlighting and colour-coding for `.sv` files, snippets, and basic linting so typos in keywords are caught before simulation.
TerosHDL	TerosHDL	A more complete HDL toolkit: linting, a signal/hierarchy viewer, and integrated waveform viewing directly inside VS Code as an alternative to launching GTKWave separately.
GitLens	GitKraken	Adds inline blame/history annotations inside files, useful once both team members start committing to the same repository.

To install any of these: open VS Code, click the Extensions icon in the left sidebar (it looks like four small squares), type the extension name into the search box, and click Install on the correct result.

> **NOTE:** Only the first extension (Verilog-HDL/SystemVerilog) is strictly required. GTKWave, launched as a separate application from the terminal, is perfectly sufficient for viewing waveforms and is what every instruction in this guide assumes. TerosHDL is an optional convenience for viewing waveforms without leaving VS Code.

4.5 Verifying the Whole Toolchain With a Trivial Circuit

Before writing a single line of the real project, it is essential to prove that the three tools actually work together. This step exists so that, later, if something goes wrong, it is known for certain that the problem is in the RISC-V code and not in the toolchain itself. This exact verification is also the first task in Chunk 1 (Section 8).

```
// File: sanity_and_gate.sv
module sanity_and_gate (
    input  logic a,
    input  logic b,
    output logic y
);
    assign y = a & b;
endmodule
```

Figure 4.1 – a two-input AND gate, the smallest possible circuit to sanity-check the tools.

```
// File: sanity_and_gate_tb.sv
`timescale 1ns/1ps
module sanity_and_gate_tb;
    logic a, b, y;

    // Instance of the circuit being tested ("Device Under Test")
    sanity_and_gate dut (.a(a), .b(b), .y(y));

    initial begin
        $dumpfile("sanity_and_gate.vcd");   // waveform output file
        $dumpvars(0, sanity_and_gate_tb);   // dump every signal in this testbench

        a = 0; b = 0; #10;
        $display("a=%0d b=%0d y=%0d", a, b, y);
        a = 0; b = 1; #10;
        $display("a=%0d b=%0d y=%0d", a, b, y);
        a = 1; b = 0; #10;
        $display("a=%0d b=%0d y=%0d", a, b, y);
        a = 1; b = 1; #10;
        $display("a=%0d b=%0d y=%0d", a, b, y);

        $finish;
    end
endmodule
```

Figure 4.2 – a testbench that drives all four input combinations and prints the output each time.

Run these two commands in the VS Code terminal, from the folder containing both files:

```
iverilog -g2012 -o sanity.vvp sanity_and_gate.sv sanity_and_gate_tb.sv
vvp sanity.vvp
```

The `-g2012` flag tells Icarus Verilog to use the 2012 SystemVerilog language standard, which is required throughout this project (plain Verilog does not have `logic`, `always_comb`, packages, or enums, all of which are used later). The expected output in the terminal is:

```
a=0 b=0 y=0
a=0 b=1 y=0
a=1 b=0 y=0
a=1 b=1 y=1
```

If exactly this appears, all three tools are correctly installed and talking to each other correctly. To also confirm GTKWave, run `gtkwave sanity_and_gate.vcd`, which opens a window; click on `a`, `b` and `y` in the left-hand signal list and drag them into the main waveform pane to see them drawn as timing waveforms over the 40 ns of simulated time.

5 Repository Structure and Git Workflow

5.1 Folder Structure

Before Chunk 1 begins, the repository should be organised as shown below. Chunk 1 (Section 8) creates these folders; this table is the reference to come back to at any point.

```
mini-riscv-instr-decoder/
|-- README.md
|-- .gitignore
|-- docs/
|   `-- project_guide.pdf        (this document)
|-- rtl/
|   |-- riscv_pkg.sv              (Chunk 2)
|   |-- field_extractor.sv        (Chunk 2)
|   |-- imm_generator.sv          (Chunks 3 and 4)
|   |-- instr_type_decoder.sv     (Chunk 4)
|   |-- control_unit.sv           (Chunk 5)
|   `-- riscv_instr_decoder.sv    (Chunk 6, top level)
`-- tb/
    |-- sanity_and_gate.sv        (Chunk 1)
    |-- sanity_and_gate_tb.sv     (Chunk 1)
    |-- tb_field_extractor.sv     (Chunk 2)
    |-- tb_imm_generator.sv       (Chunks 3 and 4)
    |-- tb_control_unit.sv        (Chunk 5)
    |-- tb_riscv_instr_decoder.sv (Chunk 6)
    `-- tb_selfcheck.sv           (Chunk 7)
```

`rtl` (register-transfer level) holds only the actual design – the modules that would still exist if this were built into a real chip. `tb` (testbench) holds only the code written purely to verify the design in simulation; none of it is ever "synthesised" into real hardware. Keeping this separation from day one is standard professional practice in digital design and also makes it obvious, at a glance, which files are safe to run standalone tests on.

5.2 Why a Working Branch

The `main` branch should, at every point in time, contain only code that both team members have personally simulated and confirmed correct. Everything experimental, in-progress, or not yet checked happens on `working` instead. This means that even if a mistake is pushed to `working`, `main` is never affected until that mistake is found, fixed, and deliberately merged across. Since the repository is already created with both branches and already linked to GitHub, no repository creation steps are needed here – only the day-to-day commands below.

5.3 The Standard Cycle For Every Single Chunk

This exact seven-step cycle repeats for all eight chunks, regardless of whose turn it is. Learning it once here means Sections 8 through 15 can simply say "follow the standard cycle" instead of repeating all seven steps eight times.

1. **Sync first.** Before starting any new work, pull the latest `working` branch so local files match what the other teammate last pushed: run `git checkout working` followed by `git pull origin working`.
2. **Do the work.** Create or edit exactly the files listed for that chunk.
3. **Simulate and verify.** Run the `iverilog` / `vvp` commands given in that chunk's section and confirm the output matches the expected output shown in this guide, line for line.
4. **Stage the changes.** Run `git add` for exactly the files that were created or changed (or `git add .` if every changed file in the folder belongs to this chunk).
5. **Commit with a clear message.** Run `git commit -m "..."` using a short message that names the chunk and what it added, for example: `git commit -m "Chunk 2: add field_extractor module and testbench"`.
6. **Push to the working branch.** Run `git push origin working`.
7. **Merge into main once verified.** After both teammates agree the chunk is correct (a quick message to each other confirming "tested, output matches" is enough for a project of this size), merge `working` into

`main`: `git checkout main`, then `git pull origin main`, then `git merge working`, then `git push origin main`. Finally switch back with `git checkout working` so the next chunk starts from the right branch.

> **NOTE:** If, at Step 1, `git pull` reports a merge conflict because both teammates edited the same file at the same time, do not panic and do not delete anything. VS Code's built-in merge editor (it opens automatically and shows “Current Change” and “Incoming Change” side by side) is the easiest way to resolve this: click “Accept Incoming”, “Accept Current”, or manually edit the file to keep the correct lines from both sides, then `git add` the resolved file and `git commit` to complete the merge. This situation is covered again, with a concrete example, in Section 19 (Troubleshooting).

5.4 A Rule That Prevents Almost All Conflicts

Because the sub-modules in this project are genuinely independent pieces of logic (Section 3.1), the two team members are almost never editing the same file in the same week. Chunk ownership already guarantees this as long as the order in Section 7 is followed – whoever is not actively working on the current chunk should avoid editing files under `rtl/` until their own chunk begins. Documentation files such as `README.md` are the one exception where both may want to add a line at different times; for those, simply pull immediately before editing and push immediately after, keeping the window for a collision as short as possible.

6 SystemVerilog Essentials Used in This Project

SystemVerilog is a large language, but this project deliberately only needs a small, beginner-safe subset of it, because the entire decoder is combinational logic with no clock. Everything in this section is everything that will be used – nothing more will appear anywhere in the code that follows.

6.1 `logic` – the one signal type needed

Older Verilog forces a choice between `wire` and `reg` depending on how a signal is assigned. SystemVerilog's `logic` type removes this confusion: it can be used almost everywhere, for both inputs, outputs, and internal signals, as long as it is only ever driven from one place. Every signal in this project is declared `logic`.

6.2 Modules and Ports

```
module module_name (
    input  logic [7:0] some_input,   // 8-bit wide input
    output logic        some_output  // 1-bit wide output
);
    // body goes here
endmodule
```

`input` and `output` declare the direction of a port. `[7:0]` after the type declares the width and bit ordering – `[7:0]` means bit 7 is the most significant (leftmost) bit and bit 0 is the least significant (rightmost) bit, which is the standard convention used throughout this project and the RISC-V specification.

6.3 `assign` and `always_comb` – describing combinational logic

There are two ways to describe a signal that is a pure function of its inputs, with no memory of the past. Both are used in this project, in different places for readability.

- `assign target = expression;` is used for a single, simple expression, for example `assign opcode = instr[6:0];`. It can appear directly in the module body, outside any `always` block.
- `always_comb begin ... end` is used when several signals need to be computed together, or when a `case` statement is needed to choose between several possibilities. Everything inside reacts instantly and automatically whenever any signal it reads changes – nothing needs to be listed manually in a sensitivity list, unlike old-style Verilog `always @(*)`.

> **NOTE:** This project never uses `always_ff` (which describes clocked, memory-holding logic) because a decoder has no concept of “the previous instruction” – its output depends only on the instruction present on its input at this instant. If `always_ff` or a clock signal ever appears necessary while writing a chunk, that is a sign something has been misunderstood; stop and re-read the relevant section.

6.4 Bit Slicing and Concatenation

Pulling out a range of bits from a wider signal is written `signal[high:low]`, for example `instr[31:25]` for the top seven bits. Joining several smaller pieces into one wider signal is written with curly braces, for example `{a, b, c}` joins `a`, `b` and `c` together with `a` becoming the most significant part. Repeating one piece several times uses the replication operator, written `{count{piece}}` – for example `{20{instr[31]}}` means “repeat the single bit `instr[31]` twenty times”, which is exactly the mechanism used for sign

extension throughout Chunks 3 and 4.

6.5 `case` Statements

```
always_comb begin
    case (opcode)
        7'b0110011: instr_type = R_TYPE;
        7'b0010011: instr_type = I_TYPE;
        default:     instr_type = INVALID;
    endcase
end
```

A `case` statement compares one signal against a list of possible exact values and executes the matching branch. `default` catches every value not explicitly listed – always include it, even when it feels unnecessary, because it prevents SystemVerilog from inferring an unwanted latch (a piece of unintended memory) for any case that is not covered.

6.6 Packages, Parameters and Enums

A `package` is a shared container for constants and type definitions that several modules need to agree on, so that a value such as “the opcode for an R-type instruction” is written once and reused everywhere by name, rather than retyped (and potentially mistyped) as a raw binary literal in five different files. A module gains access to everything inside a package with an `import` statement placed just before the module declaration.

```
package riscv_pkg;
    parameter logic [6:0] OPCODE_RTYPE = 7'b0110011;

    typedef enum logic [2:0] {
        R_TYPE, I_TYPE, S_TYPE, B_TYPE, U_TYPE, J_TYPE, INVALID
    } instr_type_e;
endpackage

import riscv_pkg::*;   // placed just before a module that needs these names

module some_module (
    output instr_type_e instr_type
);
    ...
endmodule
```

An `enum` (enumerated type) gives a small named set of values to a signal instead of forcing it to be raw binary. `instr_type_e` above is exactly the type used to carry “which of the six formats is this” around the design in Chunk 4 – far more readable in code and in a GTKWave waveform than a meaningless 3-bit number would be.

6.7 Number Literals

SystemVerilog writes sized, based numbers as `'`, for example `7'b0110011` (7 bits, binary), `12'd10` (12 bits, decimal 10), or `32'h003100B3` (32 bits, hexadecimal). This project uses `b` (binary) for opcode/funct3/funct7 constants, since binary makes the exact bit pattern obvious, and `h` (hexadecimal) for whole 32-bit test instructions in testbenches, since hexadecimal is far more compact to read and type.

6.8 Testbench-Only Constructs

A handful of extra constructs are used only inside testbench files, never inside real design files, because they have no equivalent in physical hardware and exist purely to drive and observe a simulation.

Construct	Purpose
`initial begin ... end`	A block that runs exactly once, starting at simulation time zero. All testbenches in this project consist of a single `initial` block.
`#10`	A delay statement meaning “let 10 simulated time units pass”. It gives the combinational logic time to settle before the result is checked.
`$display("...", signal);`	Prints a formatted line of text to the terminal immediately, once.
`$dumpfile("name.vcd") / $dumpvars(0, top);`	Tells the simulator to record every signal's value over time into a waveform file that GTKWave can later open.
`$finish;`	Ends the simulation cleanly.
`$error("...");`	Prints a message flagged as an error; used in the self-checking testbench in Chunk 7 to make a mismatch impossible to miss.

7 Implementation Plan – The Eight Chunks at a Glance

The project is split into eight sequential chunks. Work always happens on the `working` branch; each chunk is merged into `main` only after it has been built and simulated successfully, following the cycle in Section 5.3. Ownership alternates strictly between the two team members, starting with Aryan, so that both people touch both the "data" side (extracting fields and immediates) and the "decision" side (classification and control) of the design.

#	Owner	Chunk title	Main file(s) created
1	Aryan	Environment Verification and Repository Skeleton	tb/sanity_and_gate*.sv, folders, README.md
2	Abhimanyu	Constants Package and Field Extractor	rtl/riscv_pkg.sv, rtl/field_extractor.sv
3	Aryan	Immediate Generator, Part 1 (I, S, U types)	rtl/imm_generator.sv
4	Abhimanyu	Immediate Generator, Part 2 (B, J) and Classifier	rtl/imm_generator.sv (extended), rtl/instr_type_decoder.sv
5	Aryan	The Control Unit	rtl/control_unit.sv
6	Abhimanyu	Top-Level Integration	rtl/riscv_instr_decoder.sv
7	Aryan	Self-Checking Testbench and Waveform Verification	tb/tb_selfcheck.sv
8	Abhimanyu	Documentation and Submission Readiness	README.md (final), cleanup

NOTE: Every chunk is independently simulatable the moment it is finished – nobody has to wait for a later chunk to test their own work. Chunks 3 and 4 both touch `imm_generator.sv`, since the immediate-generation logic is naturally one module; Chunk 4 simply adds two more branches to the `case` statement Chunk 3 started, which is explained in full in Section 11.

8 Chunk 1 – Environment Verification and Repository Skeleton

CHUNK OWNER : ARYAN

8.1 Objective

Prove that the toolchain (VS Code, Icarus Verilog, GTKWave) genuinely works end to end, and lay out the folder structure the rest of the project will live in, before any real RISC-V code is written. Doing this first means that if anything ever fails to compile later, the toolchain itself can immediately be ruled out as the cause.

8.2 Steps

1. Inside the already-cloned repository, on the `working` branch, create three empty folders named `rtl`, `tb` and `docs`. In VS Code's Explorer panel this can be done with right-click → "New Folder", or from the terminal with `mkdir rtl tb docs`.
2. Create the two files shown in Section 4.5 of this guide (Figures 4.1 and 4.2), `sanity_and_gate.sv` and `sanity_and_gate_tb.sv`, inside the `tb` folder.
3. Open a terminal in the repository's root folder and compile and run exactly as shown in Section 4.5: `iverilog -g2012 -o sanity.vvp tb/sanity_and_gate.sv tb/sanity_and_gate_tb.sv` followed by `vvp sanity.vvp`. Confirm the four lines of output match Section 4.5 exactly.
4. Create a file named `.gitignore` in the repository root (see Section 8.3 below for its contents). This stops simulator-generated clutter from ever being committed to Git.
5. Create `README.md` in the repository root with the starter content given in Section 8.4. It will be filled in further with each chunk and finalised in Chunk 8.
6. Follow the standard cycle from Section 5.3: stage everything with `git add .`, commit with `git commit -m "Chunk 1: verify toolchain, add repo skeleton"`, push with `git push origin working`, confirm with Abhimanyu, then merge into `main`.

8.3 Contents of `.gitignore`

```
# Simulator build/output artefacts - never commit these
*.vvp
*.vcd
*.out

# Editor/OS clutter
.vscode/
Thumbs.db
.DS_Store
```

`.vvp` files are compiled simulation binaries and `.vcd` files are waveform dumps – both can always be regenerated from the source `.sv` files in seconds, so keeping them out of Git keeps the repository small and avoids meaningless "changed" diffs on files that are not really source code.

8.4 Starter Contents of `README.md`

```
# Mini RISC-V Instruction Decoder

A SystemVerilog implementation of a combinational instruction decoder for a
subset of the RV32I base integer instruction set, built as a final-year
B.Tech ECE mini-project.

## Team
- Aryan
- Abhimanyu

## Status
Project skeleton created. Toolchain (Icarus Verilog + GTKWave) verified.

## How to simulate
See docs/project_guide.pdf for full instructions.
```

8.5 Definition of Done

- [ ] `iverilog -V` and `gtkwave --version` both print a version with no errors, on both team members' machines.
- [ ] The sanity AND-gate testbench produces exactly the four output lines shown in Section 4.5.
- [ ] The folders `rtl/`, `tb/` and `docs/` exist in the repository.
- [ ] `.gitignore` and a starter `README.md` are committed.
- [ ] The chunk has been pushed to `working` and merged into `main`.

9 Chunk 2 – Constants Package and the Field Extractor

CHUNK OWNER : ABHIMANYU

9.1 Objective

Create the shared constants package that every later module will import, and build the first real piece of the decoder: a module that slices the fixed-position fields (opcode, rd, funct3, rs1, rs2, funct7) out of a 32-bit instruction, exactly as introduced in Section 2.5.

9.2 File: `rtl/riscv_pkg.sv`

```
package riscv_pkg;

    // ------------------------------------------------------------------
    // Opcode values (instruction bits [6:0])
    // ------------------------------------------------------------------
    parameter logic [6:0] OPCODE_RTYPE = 7'b0110011; // add, sub, and, or, xor, slt
    parameter logic [6:0] OPCODE_ITYPE = 7'b0010011; // addi, andi, ori
    parameter logic [6:0] OPCODE_LOAD  = 7'b0000011; // lw
    parameter logic [6:0] OPCODE_STYPE = 7'b0100011; // sw
    parameter logic [6:0] OPCODE_BTYPE = 7'b1100011; // beq, bne
    parameter logic [6:0] OPCODE_UTYPE = 7'b0110111; // lui
    parameter logic [6:0] OPCODE_JTYPE = 7'b1101111; // jal

    // ------------------------------------------------------------------
    // funct3 values (instruction bits [14:12])
    // ------------------------------------------------------------------
    parameter logic [2:0] F3_ADD_SUB = 3'b000;
    parameter logic [2:0] F3_SLT     = 3'b010;
    parameter logic [2:0] F3_XOR     = 3'b100;
    parameter logic [2:0] F3_OR      = 3'b110;
    parameter logic [2:0] F3_AND     = 3'b111;
    parameter logic [2:0] F3_LW_SW   = 3'b010;
    parameter logic [2:0] F3_BEQ     = 3'b000;
    parameter logic [2:0] F3_BNE     = 3'b001;

    // ------------------------------------------------------------------
    // funct7 values (instruction bits [31:25], R-type only)
    // ------------------------------------------------------------------
    parameter logic [6:0] F7_NORMAL = 7'b0000000; // add, and, or, xor, slt
    parameter logic [6:0] F7_SUB    = 7'b0100000; // sub

    // ------------------------------------------------------------------
    // ALU control codes - tell a (future) ALU exactly what to compute
    // ------------------------------------------------------------------
    parameter logic [3:0] ALU_ADD = 4'b0000;
    parameter logic [3:0] ALU_SUB = 4'b0001;
    parameter logic [3:0] ALU_AND = 4'b0010;
    parameter logic [3:0] ALU_OR  = 4'b0011;
    parameter logic [3:0] ALU_XOR = 4'b0100;
    parameter logic [3:0] ALU_SLT = 4'b0101;

    // ------------------------------------------------------------------
    // Instruction type classification (built fully in Chunk 4)
    // ------------------------------------------------------------------
    typedef enum logic [2:0] {
        R_TYPE  = 3'd0,
        I_TYPE  = 3'd1,
        S_TYPE  = 3'd2,
        B_TYPE  = 3'd3,
        U_TYPE  = 3'd4,
        J_TYPE  = 3'd5,
        INVALID = 3'd6
    } instr_type_e;

endpackage
```

Figure 9.1 – rtl/riscv_pkg.sv, the single source of truth for every named constant used in the project.

> **NOTE:** Every value here was already introduced in Table 3.1. Typing them once, by name, into this package – rather than typing raw binary numbers again in every module – is what stops a typo in one file from silently breaking a different file.

9.3 File: `rtl/field_extractor.sv`

```
module field_extractor (
    input  logic [31:0] instr,
    output logic [6:0]  opcode,
    output logic [4:0]  rd,
    output logic [2:0]  funct3,
    output logic [4:0]  rs1,
    output logic [4:0]  rs2,
    output logic [6:0]  funct7
);
    assign opcode = instr[6:0];
    assign rd     = instr[11:7];
    assign funct3 = instr[14:12];
    assign rs1    = instr[19:15];
    assign rs2    = instr[24:20];
    assign funct7 = instr[31:25];
endmodule
```

Figure 9.2 – rtl/field_extractor.sv, pure bit-slicing with no case statement needed.

This module always extracts all six fields from the same six bit ranges, regardless of what kind of instruction is actually present. This is intentional and mirrors real hardware: the wires from these fixed bit positions run into the rest of the processor unconditionally, and it is the job of later modules (the immediate generator and the control unit) to decide which of these extracted values actually matter for a given instruction.

> **NOTE:** A consequence worth understanding now rather than being confused by later: for an instruction format that does not really have an `rs1`, `rs2` or `funct3` – for example LUI, which is U-type – this module will still output *something* on those pins, because it has no way of knowing those bit positions are not meaningful for this particular opcode. That output is simply meaningless / "don't care" for that instruction, and every testbench in this guide that uses a U-type or J-type instruction explicitly ignores those particular signals for exactly this reason.

9.4 Test Plan

The testbench applies four instructions from Table 3.1, chosen to cover an R-type, an I-type, an S-type and a U-type instruction, and checks only the fields that are meaningful for each. All four hexadecimal values below were derived and cross-checked in Section 2.7 and Table 3.1's format.

Instruction	Hex	opcode	rd	funct3	rs1	rs2	funct7
ADD x1,x2,x3	0x003100B3	0110011	1	000	2	3	0000000
ADDI x5,x6,10	0x00A30293	0010011	5	000	6	–	–
SW x3,12(x4)	0x00322623	0100011	–	010	4	3	–
LUI x5,0x12345	0x123452B7	0110111	5	–	–	–	–

Caption – Table 9.1: expected field-extractor output for the four test instructions. A dash means "not meaningful for this instruction, do not check", per the note above.

9.5 File: `tb/tb_field_extractor.sv`

```
`timescale 1ns/1ps
module tb_field_extractor;
    logic [31:0] instr;
    logic [6:0]  opcode, funct7;
    logic [4:0]  rd, rs1, rs2;
    logic [2:0]  funct3;

    field_extractor dut (
        .instr  (instr),
        .opcode (opcode),
        .rd     (rd),
        .funct3 (funct3),
        .rs1    (rs1),
        .rs2    (rs2),
        .funct7 (funct7)
    );

    task apply(input [31:0] i, input string name);
        instr = i;
        #10;
        $display("%-16s instr=%h opcode=%b rd=%0d funct3=%b rs1=%0d rs2=%0d funct7=%b",
                 name, instr, opcode, rd, funct3, rs1, rs2, funct7);
    endtask

    initial begin
        $dumpfile("tb_field_extractor.vcd");
        $dumpvars(0, tb_field_extractor);

        apply(32'h003100B3, "ADD x1,x2,x3");
        apply(32'h00A30293, "ADDI x5,x6,10");
        apply(32'h00322623, "SW x3,12(x4)");
        apply(32'h123452B7, "LUI x5,0x12345");

        $finish;
    end
endmodule
```

Figure 9.3 – tb/tb_field_extractor.sv

Compile and run with:

```
iverilog -g2012 -o fe.vvp rtl/field_extractor.sv tb/tb_field_extractor.sv
vvp fe.vvp
```

Expected terminal output (compare every field against Table 9.1; ignore the dashed / not-applicable fields for ADDI, SW and LUI as explained above):

```
ADD x1,x2,x3     instr=003100b3 opcode=0110011 rd=1 funct3=000 rs1=2 rs2=3 funct7=0000000
ADDI x5,x6,10    instr=00a30293 opcode=0010011 rd=5 funct3=000 rs1=6 rs2=10 funct7=0000000
SW x3,12(x4)     instr=00322623 opcode=0100011 rd=12 funct3=010 rs1=4 rs2=3 funct7=0000000
LUI x5,0x12345   instr=123452b7 opcode=0110111 rd=5 funct3=101 rs1=8 rs2=3 funct7=0001001
```

> **NOTE:** Notice `rd=10` for the ADDI line, `rd=12` for the SW line, and non-zero funct3/rs1/rs2/funct7 for the LUI line – none of these match what the instruction “means” in assembly. This is expected and is exactly the “don't care” behaviour explained in Section 9.3: those bit positions are being read correctly, they simply do not carry `rd`/`rs1`/etc. information for these particular opcodes. Only the fields marked with an actual value in Table 9.1 need to be checked.

9.6 Definition of Done

- [ ] `rtl/riscv_pkg.sv` compiles with no errors on its own (`iverilog -g2012 -t null rtl/riscv_pkg.sv`).
- [ ] `rtl/field_extractor.sv` produces exactly the four lines shown in Section 9.5.
- [ ] Every field marked with a real value in Table 9.1 matches the simulator output exactly.
- [ ] Changes committed and pushed to `working` with a message such as "Chunk 2: add riscv_pkg and field_extractor", then merged into `main` after Aryan confirms the output.

10 Chunk 3 – Immediate Generator, Part 1 (I, S and U Types)

CHUNK OWNER : ARYAN

10.1 Objective

Build the module that reconstructs the correct signed 32-bit immediate value from an instruction, starting with the three easier formats: I-type, S-type and U-type. B-type and J-type, which need their bits reordered as well as sign-extended, are added on top of this same file in Chunk 4 (Section 11), so that the harder bit-shuffling logic is tackled separately.

10.2 Recap of the Formulas (from Section 2.6)

Format	SystemVerilog expression for the 32-bit immediate
I-type	`{{20{instr[31]}}, instr[31:20]}`
S-type	`{{20{instr[31]}}, instr[31:25], instr[11:7]}`
U-type	`{instr[31:12], 12'b0}`

I-type simply sign-extends the contiguous 12-bit field at the top of the instruction. S-type reassembles its two separated pieces (`instr[31:25]` as the upper 7 bits, `instr[11:7]` as the lower 5 bits) before sign-extending. U-type needs no sign extension at all in the traditional sense – the top 20 bits are taken exactly as they appear and the bottom 12 bits are simply set to zero, since LUI is defined to place its immediate directly into the upper 20 bits of the result.

10.3 File: `rtl/imm_generator.sv` (first version)

```
import riscv_pkg::*;

module imm_generator (
    input  logic [31:0] instr,
    output logic [31:0] imm
);

    logic [6:0] opcode;
    assign opcode = instr[6:0];

    always_comb begin
        case (opcode)

            OPCODE_ITYPE, OPCODE_LOAD: begin
                // I-type: imm[11:0] = instr[31:20], sign-extended
                imm = {{20{instr[31]}}, instr[31:20]};
            end

            OPCODE_STYPE: begin
                // S-type: imm[11:5] = instr[31:25], imm[4:0] = instr[11:7]
                imm = {{20{instr[31]}}, instr[31:25], instr[11:7]};
            end

            OPCODE_UTYPE: begin
                // U-type: imm[31:12] = instr[31:12], lower 12 bits = 0
                imm = {instr[31:12], 12'b0};
            end

            default: imm = 32'd0;   // R-type has no immediate; B/J added in Chunk 4

        endcase
    end

endmodule
```

Figure 10.1 – rtl/imm_generator.sv. The B_TYPE and J_TYPE branches are added inside this same case statement in Chunk 4; for now they simply fall into `default` and correctly output zero.

NOTE: `OPCODE_ITYPE, OPCODE_LOAD:` on one line means "run this same branch whether the opcode is OPCODE_ITYPE or OPCODE_LOAD" – both use an identical immediate layout, so there is no need to repeat the line twice. This comma-separated case-item syntax is used again in Chunk 5.

10.4 Test Plan

Six instructions from Table 3.1 are used, chosen specifically so that every format handled so far is tested with both a positive and, where meaningful, a negative immediate – negative values are the case most likely to expose a sign-extension mistake.

Instruction	Hex	Expected imm (decimal)	Expected imm (hex)
ADDI x5,x6,10	0x00A30293	10	0x0000000A
ADDI x5,x6,-5	0xFFB30293	-5	0xFFFFFFFB
LW x1,8(x2)	0x00812083	8	0x00000008
LW x9,-4(x10)	0xFFC52483	-4	0xFFFFFFFC
SW x3,12(x4)	0x00322623	12	0x0000000C
SW x8,-8(x9)	0xFE84AC23	-8	0xFFFFFFF8
LUI x5,0x12345	0x123452B7	305418240	0x12345000

Caption – Table 10.1: every value here was generated and cross-checked with an independent Python reference model before being placed in this guide, so a mismatch during simulation reliably points to a mistake in the SystemVerilog rather than in the expected value.

10.5 File: `tb/tb_imm_generator.sv` (first version)

```
`timescale 1ns/1ps
module tb_imm_generator;
    logic [31:0] instr;
    logic [31:0] imm;

    imm_generator dut (.instr(instr), .imm(imm));

    task apply(input [31:0] i, input string name, input int expected);
        instr = i;
        #10;
        if (imm === expected)
            $display("PASS  %-16s imm=%0d (0x%08h)", name, imm, imm);
        else
            $display("FAIL  %-16s imm=%0d, expected=%0d", name, $signed(imm), expected);
    endtask

    initial begin
        $dumpfile("tb_imm_generator.vcd");
        $dumpvars(0, tb_imm_generator);

        apply(32'h00A30293, "ADDI +10",   10);
        apply(32'hFFB30293, "ADDI -5",    -5);
        apply(32'h00812083, "LW +8",       8);
        apply(32'hFFC52483, "LW -4",      -4);
        apply(32'h00322623, "SW +12",     12);
        apply(32'hFE84AC23, "SW -8",      -8);
        apply(32'h123452B7, "LUI",         32'h12345000);

        $finish;
    end
endmodule
```

Figure 10.2 – tb/tb_imm_generator.sv. Note the `task` compares against an expected value and prints PASS or FAIL directly – this is the first self-checking testbench in the project, a style that is expanded fully in Chunk 7.

> **NOTE:** `$signed(imm)` is used only inside the FAIL message so that a negative expected value is printed as a negative number instead of as a huge unsigned one, purely to make a failure easier to read. The comparison itself, `imm === expected`, works correctly either way because `expected` is declared as a signed `int`, which SystemVerilog compares bit-for-bit against the 32-bit `imm`.

Compile and run with:

```
iverilog -g2012 -o ig.vvp rtl/riscv_pkg.sv rtl/imm_generator.sv tb/tb_imm_generator.sv
vvp ig.vvp
```

Expected output – every line should say PASS:

```
PASS  ADDI +10        imm=10 (0x0000000a)
PASS  ADDI -5         imm=4294967291 (0xfffffffb)
PASS  LW +8           imm=8 (0x00000008)
PASS  LW -4           imm=4294967292 (0xfffffffc)
PASS  SW +12          imm=12 (0x0000000c)
PASS  SW -8           imm=4294967288 (0xfffffff8)
PASS  LUI             imm=305418240 (0x12345000)
```

NOTE: `imm` is declared as plain (unsigned) `logic [31:0]`, so `$display`'s `%0d` prints negative results as their large unsigned equivalent (for example 4294967291 instead of -5) – both represent exactly the same 32 bits, and this is exactly why the FAIL branch above uses `$signed()` for readability. What matters is that the PASS/FAIL comparison itself, done on the raw bits, is correct; Section 16 explains two's-complement representation in more depth for the write-up.

10.6 Definition of Done

- [ ] All seven lines of simulator output say PASS, exactly matching Table 10.1.
- [ ] The default branch is confirmed to still produce `imm = 0` for an R-type instruction (optionally add an eighth test line using the ADD instruction from Table 3.1 to double check this).
- [ ] Changes committed and pushed to `working`, then merged into `main` once Abhimanyu confirms the output.

11 Chunk 4 – Immediate Generator Part 2 (B, J) and Instruction Classifier

CHUNK OWNER : ABHIMANYU

11.1 Objective

Finish `imm_generator.sv` by adding the two trickiest formats, B-type and J-type, whose immediate bits are stored out of numerical order (Sections 2.5.4 and 2.5.6). Then build a small new module, `instr_type_decoder`, that looks only at the opcode and reports which of the six formats an instruction belongs to – useful on its own for debugging in GTKWave, and used again inside the top-level module in Chunk 6.

11.2 Recap of the Formulas (from Section 2.6)

Format	SystemVerilog expression for the 32-bit immediate
B-type	`{{19{instr[31]}}, instr[31], instr[7], instr[30:25], instr[11:8], 1'b0}`
J-type	`{{11{instr[31]}}, instr[31], instr[19:12], instr[20], instr[30:21], 1'b0}`

Reading the B-type expression from left to right: 19 copies of the sign bit, then the sign bit itself again (this is `imm[12]` from the format diagram), then `instr[7]` (which is `imm[11]`), then `instr[30:25]` (`imm[10:5]`), then `instr[11:8]` (`imm[4:1]`), then a literal zero bit for the implicit least-significant bit that branch targets never store. Counting the bits confirms the total is exactly 32: 19 + 1 + 1 + 6 + 4 + 1 = 32. The J-type expression follows the identical idea over its own scrambled layout, totalling 11 + 1 + 8 + 1 + 10 + 1 = 32.

> **NOTE:** It is completely normal for this concatenation order to look unintuitive on first reading – it is not something to derive from first principles under exam pressure, it is something to look up (from the RISC-V specification, or from this table) and apply carefully, bit count by bit count, the same way these two expressions were built for this guide.

11.3 File: `rtl/imm_generator.sv` (final version – edit the Chunk 3 file)

```
import riscv_pkg::*;

module imm_generator (
    input  logic [31:0] instr,
    output logic [31:0] imm
);

    logic [6:0] opcode;
    assign opcode = instr[6:0];

    always_comb begin
        case (opcode)

            OPCODE_ITYPE, OPCODE_LOAD: begin
                imm = {{20{instr[31]}}, instr[31:20]};
            end

            OPCODE_STYPE: begin
                imm = {{20{instr[31]}}, instr[31:25], instr[11:7]};
            end

            OPCODE_BTYPE: begin
                // B-type: imm[12|11|10:5|4:1], implicit 0 in bit 0
                imm = {{19{instr[31]}}, instr[31], instr[7], instr[30:25], instr[11:8], 1'b0};
            end

            OPCODE_UTYPE: begin
                imm = {instr[31:12], 12'b0};
            end

            OPCODE_JTYPE: begin
                // J-type: imm[20|19:12|11|10:1], implicit 0 in bit 0
                imm = {{11{instr[31]}}, instr[31], instr[19:12], instr[20], instr[30:21], 1'b0};
            end

            default: imm = 32'd0;   // R-type: no immediate

        endcase
    end

endmodule
```

Figure 11.1 – rtl/imm_generator.sv, final version. Only the two new `OPCODE_BTYPE` and `OPCODE_JTYPE` branches were added to the file from Chunk 3; everything else is unchanged.

11.4 File: `rtl/instr_type_decoder.sv`

```
import riscv_pkg::*;

module instr_type_decoder (
    input  logic [31:0]  instr,
    output instr_type_e  instr_type
);
    logic [6:0] opcode;
    assign opcode = instr[6:0];

    always_comb begin
        case (opcode)
            OPCODE_RTYPE:                instr_type = R_TYPE;
            OPCODE_ITYPE, OPCODE_LOAD:   instr_type = I_TYPE;
            OPCODE_STYPE:                instr_type = S_TYPE;
            OPCODE_BTYPE:                instr_type = B_TYPE;
            OPCODE_UTYPE:                instr_type = U_TYPE;
            OPCODE_JTYPE:                instr_type = J_TYPE;
            default:                     instr_type = INVALID;
        endcase
    end
endmodule
```

Figure 11.2 – rtl/instr_type_decoder.sv. This module reuses the `instr_type_e` enum already defined in riscv_pkg.sv (Figure 9.1) so no new type needs to be introduced here.

11.5 Test Plan and Expected Values

The immediate-generator testbench from Chunk 3 is extended with four more lines covering BEQ, BNE and both JAL cases (including `JAL x0, -50`, deliberately chosen because it writes to `x0` and uses a negative offset at the same time – a good combined edge case).

Instruction	Hex	Expected imm (decimal)	Expected imm (hex)
BEQ x1,x2,8	0x00208463	8	0x00000008
BNE x1,x2,-4	0xFE209EE3	-4	0xFFFFFFFC
JAL x1,100	0x064000EF	100	0x00000064
JAL x0,-50	0xFCFFF06F	-50	0xFFFFFFCE

```
// --- add these lines into the initial block of tb/tb_imm_generator.sv, before $finish ---
apply(32'h00208463, "BEQ +8",   8);
apply(32'hFE209EE3, "BNE -4",  -4);
apply(32'h064000EF, "JAL +100", 100);
apply(32'hFCFFF06F, "JAL -50", -50);
```

Figure 11.3 – additional test vectors appended to the existing testbench file.

Re-run the same two commands from Section 10.5. All eleven lines (seven from Chunk 3 plus these four) should now say PASS.

11.6 File: `tb/tb_instr_type_decoder.sv`

```
`timescale 1ns/1ps
import riscv_pkg::*;
module tb_instr_type_decoder;
    logic [31:0] instr;
    instr_type_e instr_type;

    instr_type_decoder dut (.instr(instr), .instr_type(instr_type));

    task apply(input [31:0] i, input string name, input instr_type_e expected);
        instr = i;
        #10;
        if (instr_type == expected)
            $display("PASS  %-16s type=%s", name, instr_type.name());
        else
            $display("FAIL  %-16s type=%s expected=%s", name, instr_type.name(), expected.name());
    endtask

    initial begin
        $dumpfile("tb_instr_type_decoder.vcd");
        $dumpvars(0, tb_instr_type_decoder);

        apply(32'h003100B3, "ADD",  R_TYPE);
        apply(32'h00A30293, "ADDI", I_TYPE);
        apply(32'h00322623, "SW",   S_TYPE);
        apply(32'h00208463, "BEQ",  B_TYPE);
        apply(32'h123452B7, "LUI",  U_TYPE);
        apply(32'h064000EF, "JAL",  J_TYPE);

        $finish;
    end
endmodule
```

Figure 11.4 – tb/tb_instr_type_decoder.sv. `.name()` is a built-in SystemVerilog enum method that converts an enum value back into its readable name for printing.

Compile and run with:

```
iverilog -g2012 -o itd.vvp rtl/riscv_pkg.sv rtl/instr_type_decoder.sv tb/tb_instr_type_decoder.sv
vvp itd.vvp
```

Expected output:

```
PASS  ADD             type=R_TYPE
PASS  ADDI            type=I_TYPE
PASS  SW              type=S_TYPE
PASS  BEQ             type=B_TYPE
PASS  LUI             type=U_TYPE
PASS  JAL             type=J_TYPE
```

11.7 Definition of Done

- [ ] All eleven lines of `tb_imm_generator` output say PASS (seven from Chunk 3, four new ones).
- [ ] All six lines of `tb_instr_type_decoder` output say PASS.
- [ ] Changes committed and pushed to `working`, then merged into `main` once Aryan confirms the output.

12 Chunk 5 – The Control Unit

CHUNK OWNER : ARYAN

12.1 Objective

Build the module that looks at `opcode`, `funct3` and `funct7` and decides how the rest of a processor should behave for this instruction: whether a register gets written, whether memory is read or written, which ALU operation to perform, and whether this instruction is a branch or a jump. This is the module that turns “what instruction is this” into “what should happen because of it” – the decision-making heart of the whole decoder.

12.2 The Eight Control Signals

Signal	Meaning when it is 1
`reg_write`	The result should be written back into register `rd`.
`mem_read`	A value should be read from data memory (load).
`mem_write`	A value should be written to data memory (store).
`mem_to_reg`	The value written to `rd` comes from memory, not from the ALU (only meaningful together with reg_write=1).
`alu_src`	The ALU's second operand is the immediate, not register `rs2`.
`branch`	This instruction is a conditional branch; the ALU is used to compare rs1 and rs2.
`jump`	This instruction is an unconditional jump (JAL).
`alu_ctrl`	A 4-bit code (not a single bit) telling the ALU exactly which operation to perform – see Figure 9.1 for the six named values.

12.3 File: `rtl/control_unit.sv`

```
import riscv_pkg::*;

module control_unit (
    input  logic [6:0] opcode,
    input  logic [2:0] funct3,
    input  logic [6:0] funct7,
    output logic       reg_write,
    output logic       mem_read,
    output logic       mem_write,
    output logic       mem_to_reg,
    output logic       alu_src,
    output logic       branch,
    output logic       jump,
    output logic [3:0] alu_ctrl
);

    always_comb begin
        // Safe defaults first - guarantees every output gets a value on
        // every path, which also avoids accidentally inferring a latch.
        reg_write  = 1'b0;
        mem_read   = 1'b0;
        mem_write  = 1'b0;
        mem_to_reg = 1'b0;
        alu_src    = 1'b0;
        branch     = 1'b0;
        jump       = 1'b0;
        alu_ctrl   = ALU_ADD;

        case (opcode)

            OPCODE_RTYPE: begin
                reg_write = 1'b1;
                alu_src   = 1'b0;                // operand 2 = rs2
                case ({funct7, funct3})
                    {F7_NORMAL, F3_ADD_SUB}: alu_ctrl = ALU_ADD;
                    {F7_SUB,    F3_ADD_SUB}: alu_ctrl = ALU_SUB;
                    {F7_NORMAL, F3_AND}:     alu_ctrl = ALU_AND;
                    {F7_NORMAL, F3_OR}:      alu_ctrl = ALU_OR;
                    {F7_NORMAL, F3_XOR}:     alu_ctrl = ALU_XOR;
                    {F7_NORMAL, F3_SLT}:     alu_ctrl = ALU_SLT;
                    default:                 alu_ctrl = ALU_ADD;
                endcase
            end

            OPCODE_ITYPE: begin
                reg_write = 1'b1;
                alu_src   = 1'b1;                // operand 2 = immediate
                case (funct3)
                    F3_ADD_SUB: alu_ctrl = ALU_ADD; // addi
                    F3_AND:     alu_ctrl = ALU_AND; // andi
                    F3_OR:      alu_ctrl = ALU_OR;  // ori
                    default:    alu_ctrl = ALU_ADD;
                endcase
            end
            // ... continued in Figure 12.2 below (LOAD, STYPE, BTYPE, UTYPE, JTYPE, default) ...
```

Figure 12.1 – rtl/control_unit.sv, part 1 of 2: module header, safe defaults, and the R-type / I-type cases.

```
            // --- continuing the same case(opcode) statement from Figure 12.1 ---

            OPCODE_LOAD: begin
                reg_write  = 1'b1;
                mem_read   = 1'b1;
                mem_to_reg = 1'b1;
                alu_src    = 1'b1;
                alu_ctrl   = ALU_ADD;             // address = rs1 + imm
            end

            OPCODE_STYPE: begin
                mem_write = 1'b1;
                alu_src   = 1'b1;
                alu_ctrl  = ALU_ADD;              // address = rs1 + imm
            end

            OPCODE_BTYPE: begin
                branch   = 1'b1;
                alu_src  = 1'b0;
                alu_ctrl = ALU_SUB;               // compare rs1 - rs2
            end

            OPCODE_UTYPE: begin
                reg_write = 1'b1;
                alu_src   = 1'b1;
                alu_ctrl  = ALU_ADD;              // rd = 0 + imm
            end

            OPCODE_JTYPE: begin
                reg_write = 1'b1;
                jump      = 1'b1;                 // alu_ctrl unused for a pure jump
            end

            default: begin
                // Unrecognised opcode: everything stays at the safe defaults above
            end

        endcase
    end

endmodule
```

Figure 12.2 – rtl/control_unit.sv, part 2 of 2: LOAD, STYPE, BTYPE, UTYPE, JTYPE and the closing default case. Figures 12.1 and 12.2 together are the complete, single file.

NOTE: The default-values-first pattern at the top of the `always_comb` block, followed by a `case` that only overrides what actually needs to change, is a standard and highly recommended style. It guarantees every output signal is driven on every possible path through the logic, which is exactly what prevents SystemVerilog from inferring an accidental latch – a subtle bug where a signal would incorrectly "remember" its previous value instead of being purely combinational.

NOTE: BEQ and BNE deliberately produce identical control signals here (`branch=1`, `alu_ctrl=ALU_SUB`). This is correct and matches how simple RISC-V datapaths are actually built: the control unit only needs to say "compare rs1 and rs2 and consider branching"; the `funct3` value (already available as a separate output from `field_extractor`, Chunk 2) is what a downstream branch-decision circuit would use to tell "branch if equal" (funct3 = 000) apart from "branch if not equal" (funct3 = 001). Building that final branch-decision circuit is outside this project's scope (Section 3.3), but recognising this is exactly the kind of insight worth mentioning in a viva.

12.4 Test Plan

Every one of the fifteen in-scope instructions from Table 3.1 is tested; a control unit is exactly the kind of module where an untested corner case is very easy to get wrong.

Instr	RegWr	MemRd	MemWr	M2Reg	ALUSrc	Branch	Jump	ALUCtrl
ADD	1	0	0	0	0	0	0	ADD
SUB	1	0	0	0	0	0	0	SUB
AND	1	0	0	0	0	0	0	AND
OR	1	0	0	0	0	0	0	OR
XOR	1	0	0	0	0	0	0	XOR
SLT	1	0	0	0	0	0	0	SLT
ADDI	1	0	0	0	1	0	0	ADD
ANDI	1	0	0	0	1	0	0	AND
ORI	1	0	0	0	1	0	0	OR
LW	1	1	0	1	1	0	0	ADD
SW	0	0	1	0	1	0	0	ADD
BEQ	0	0	0	0	0	1	0	SUB
BNE	0	0	0	0	0	1	0	SUB
LUI	1	0	0	0	1	0	0	ADD
JAL	1	0	0	0	0	0	1	ADD*

*Caption – Table 12.1: expected control signals for all fifteen in-scope instructions. *For JAL, `alu_ctrl` is left at its default value and is not actually used by the rest of a processor, since a jump's destination is computed from the program counter and immediate, not the ALU.*

12.5 File: `tb/tb_control_unit.sv`

```
`timescale 1ns/1ps
import riscv_pkg::*;
module tb_control_unit;
    logic [6:0] opcode, funct7;
    logic [2:0] funct3;
    logic reg_write, mem_read, mem_write, mem_to_reg, alu_src, branch, jump;
    logic [3:0] alu_ctrl;

    control_unit dut (.*);   // .* auto-connects all ports to identically-named signals

    task apply(input [6:0] op, input [2:0] f3, input [6:0] f7, input string name);
        opcode = op; funct3 = f3; funct7 = f7;
        #10;
        $display("%-6s rw=%0d mr=%0d mw=%0d m2r=%0d asrc=%0d br=%0d jmp=%0d alu=%0d",
                  name, reg_write, mem_read, mem_write, mem_to_reg,
                  alu_src, branch, jump, alu_ctrl);
    endtask

    initial begin
        $dumpfile("tb_control_unit.vcd");
        $dumpvars(0, tb_control_unit);

        apply(OPCODE_RTYPE, F3_ADD_SUB, F7_NORMAL, "ADD");
        apply(OPCODE_RTYPE, F3_ADD_SUB, F7_SUB,    "SUB");
        apply(OPCODE_RTYPE, F3_AND,     F7_NORMAL, "AND");
        apply(OPCODE_RTYPE, F3_OR,      F7_NORMAL, "OR");
        apply(OPCODE_RTYPE, F3_XOR,     F7_NORMAL, "XOR");
        apply(OPCODE_RTYPE, F3_SLT,     F7_NORMAL, "SLT");
        apply(OPCODE_ITYPE, F3_ADD_SUB, 7'b0,      "ADDI");
        apply(OPCODE_ITYPE, F3_AND,     7'b0,      "ANDI");
        apply(OPCODE_ITYPE, F3_OR,      7'b0,      "ORI");
        apply(OPCODE_LOAD,  F3_LW_SW,   7'b0,      "LW");
        apply(OPCODE_STYPE, F3_LW_SW,   7'b0,      "SW");
        apply(OPCODE_BTYPE, F3_BEQ,     7'b0,      "BEQ");
        apply(OPCODE_BTYPE, F3_BNE,     7'b0,      "BNE");
        apply(OPCODE_UTYPE, 3'b0,       7'b0,      "LUI");
        apply(OPCODE_JTYPE, 3'b0,       7'b0,      "JAL");

        $finish;
    end
endmodule
```

Figure 12.2 – tb/tb_control_unit.sv. `alu` in the printed output is `alu_ctrl` printed as a plain number (0=ADD, 1=SUB, 2=AND, 3=OR, 4=XOR, 5=SLT, per Figure 9.1).

Compile and run with:

```
iverilog -g2012 -o cu.vvp rtl/riscv_pkg.sv rtl/control_unit.sv tb/tb_control_unit.sv
vvp cu.vvp
```

Compare every line of output against Table 12.1 (recall alu_ctrl 0=ADD, 1=SUB, 2=AND, 3=OR, 4=XOR, 5=SLT).

12.6 Definition of Done

- [ ] All fifteen printed lines match Table 12.1 exactly, field by field.
- [ ] The reasoning behind the BEQ/BNE and JAL notes in Section 12.3 can be explained out loud, since these are the two rows most likely to come up as viva questions.
- [ ] Changes committed and pushed to `working`, then merged into `main` once Abhimanyu confirms the output.

13 Chunk 6 – Top-Level Integration

CHUNK OWNER : ABHIMANYU

13.1 Objective

Wire the four sub-modules built in Chunks 2, 3, 4 and 5 together into a single top-level module, `riscv_instr_decoder`, which is the actual deliverable of the whole project: one module that takes a 32-bit instruction in and produces every decoded field, the immediate, the instruction type, and every control signal out, all in one place.

13.2 How the Pieces Connect

All four sub-modules take the same 32-bit `instr` as an input (or, for `control_unit`, the fields already extracted from it), and none of them depend on each other's outputs – they all run independently and in parallel on the same instruction, which is exactly why they could be built and tested in any order across four separate chunks. The top-level module's only job is instancing all four and connecting wires between them; it contains no logic of its own.

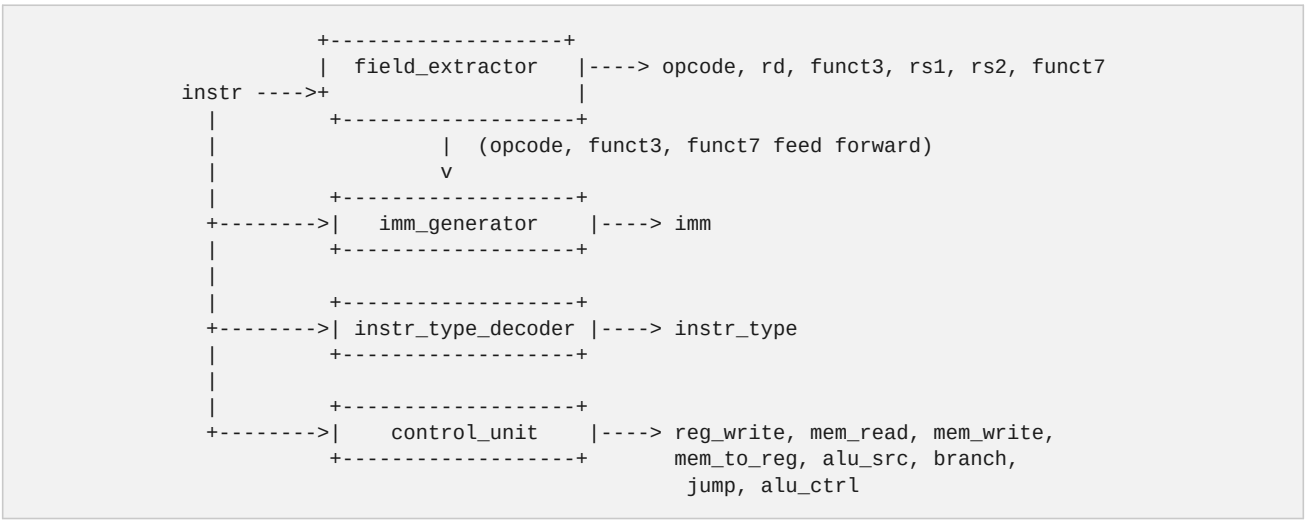


Figure 13.1 – block diagram of riscv_instr_decoder. All four sub-modules run in parallel off the same instr input; control_unit additionally needs opcode/funct3/funct7, which field_extractor has already produced.

13.3 File: `rtl/riscv_instr_decoder.sv`

```
import riscv_pkg::*;

module riscv_instr_decoder (
    input  logic [31:0]  instr,

    output logic [6:0]   opcode,
    output logic [4:0]   rd,
    output logic [2:0]   funct3,
    output logic [4:0]   rs1,
    output logic [4:0]   rs2,
    output logic [6:0]   funct7,
    output logic [31:0]  imm,
    output instr_type_e  instr_type,

    output logic reg_write,
    output logic mem_read,
    output logic mem_write,
    output logic mem_to_reg,
    output logic alu_src,
    output logic branch,
    output logic jump,
    output logic [3:0] alu_ctrl
);

    field_extractor u_field_extractor (
        .instr  (instr),
        .opcode (opcode),
        .rd     (rd),
        .funct3 (funct3),
        .rs1    (rs1),
        .rs2    (rs2),
        .funct7 (funct7)
    );

    imm_generator u_imm_generator (
        .instr (instr),
        .imm   (imm)
    );

    instr_type_decoder u_instr_type_decoder (
        .instr      (instr),
        .instr_type (instr_type)
    );

    control_unit u_control_unit (
        .opcode     (opcode),
        .funct3     (funct3),
        .funct7     (funct7),
        .reg_write  (reg_write),
        .mem_read   (mem_read),
        .mem_write  (mem_write),
        .mem_to_reg (mem_to_reg),
        .alu_src    (alu_src),
        .branch     (branch),
        .jump       (jump),
        .alu_ctrl   (alu_ctrl)
    );

endmodule
```

Figure 13.2 – rtl/riscv_instr_decoder.sv, the complete top-level decoder.

NOTE: Notice `u_control_unit` connects its `opcode`, `funct3` and `funct7` inputs to the very same signals that `u_field_extractor` produces as outputs, one instance above it. This is standard “structural” SystemVerilog: a signal declared as an output port of the top module (like `opcode`) can simultaneously be used as an input to another instance inside that same module. This is exactly the connection shown as the dashed feed-forward arrow in Figure 13.1.

13.4 Test Plan

Every one of the fifteen instructions from Table 3.1 is applied to the complete top-level module in one combined testbench, and the decoded output is printed on one line per instruction. Rather than reproducing all fifteen expected rows again here, this test deliberately re-uses the exact hex values already verified individually in Chunks 2 through 5 (Tables 9.1, 10.1, 11.5 and 12.1) – if every sub-module chunk already passed its own tests, this integration test is really checking one thing only: that the wiring in Figure 13.2 itself is correct.

13.5 File: `tb/tb_riscv_instr_decoder.sv`

```
`timescale 1ns/1ps
import riscv_pkg::*;
module tb_riscv_instr_decoder;
    logic [31:0]  instr;
    logic [6:0]   opcode, funct7;
    logic [4:0]   rd, rs1, rs2;
    logic [2:0]   funct3;
    logic [31:0]  imm;
    instr_type_e  instr_type;
    logic reg_write, mem_read, mem_write, mem_to_reg, alu_src, branch, jump;
    logic [3:0] alu_ctrl;

    riscv_instr_decoder dut (.*);

    task apply(input [31:0] i, input string name);
        instr = i;
        #10;
        $display("%-14s type=%-7s rd=%0d rs1=%0d rs2=%0d imm=%0d",
                 name, instr_type.name(), rd, rs1, rs2, $signed(imm));
        $display("               rw=%0d mr=%0d mw=%0d m2r=%0d asrc=%0d br=%0d jmp=%0d alu=%0d",
                 reg_write, mem_read, mem_write, mem_to_reg, alu_src, branch, jump, alu_ctrl);
    endtask

    initial begin
        $dumpfile("tb_riscv_instr_decoder.vcd");
        $dumpvars(0, tb_riscv_instr_decoder);
```

Figure 13.3 – tb/tb_riscv_instr_decoder.sv, part 1 of 2: declarations and the reusable `apply` task.

```
        apply(32'h003100B3, "ADD");
        apply(32'h40628233, "SUB");
        apply(32'h009473B3, "AND");
        apply(32'h00C5E533, "OR");
        apply(32'h00F746B3, "XOR");
        apply(32'h0128A833, "SLT");
        apply(32'h00A30293, "ADDI");
        apply(32'h00F1F113, "ANDI");
        apply(32'h0081E113, "ORI");
        apply(32'h00812083, "LW");
        apply(32'h00322623, "SW");
        apply(32'h00208463, "BEQ");
        apply(32'hFE209EE3, "BNE");
        apply(32'h123452B7, "LUI");
        apply(32'h064000EF, "JAL");

        $finish;
    end
endmodule
```

Figure 13.4 – tb/tb_riscv_instr_decoder.sv, part 2 of 2: all fifteen in-scope instructions from Table 3.1, applied inside the same `initial` block as Figure 13.3.

Compile and run with:

```
iverilog -g2012 -o top.vvp rtl/riscv_pkg.sv rtl/field_extractor.sv rtl/imm_generator.sv ^
       rtl/instr_type_decoder.sv rtl/control_unit.sv rtl/riscv_instr_decoder.sv ^
       tb/tb_riscv_instr_decoder.sv
vvp top.vvp
```

> **NOTE:** The `^` at the end of a line is how Windows Command Prompt (and PowerShell, where it also works) continues a long command onto the next line for readability. It can equally well be typed as one single unbroken line with spaces instead of line breaks – both behave identically.

Every value on every line can be checked against the individual chunk tables already produced (Tables 9.1, 10.1, 11.5 and 12.1); since these fifteen instructions and their expected values have already been fully worked through, this run is the moment all of that earlier work is confirmed to combine correctly into one working decoder.

13.6 Definition of Done

- [ ] The design compiles as one whole (all five `rtl/*.sv` files plus the testbench) with zero errors or warnings.
- [ ] All fifteen printed lines are individually cross-checked against the values already established in Tables 9.1, 10.1, 11.5 and 12.1.
- [ ] The waveform `tb_riscv_instr_decoder.vcd` opens correctly in GTKWave and `instr_type` visibly changes between R_TYPE, I_TYPE, S_TYPE, B_TYPE, U_TYPE and J_TYPE across the fifteen applied instructions.
- [ ] Changes committed and pushed to `working`, then merged into `main` once Aryan confirms the output.

14 Chunk 7 – Self-Checking Testbench and Waveform Verification

CHUNK OWNER : ARYAN

14.1 Objective

Replace “eyeball every printed line against a table” (which is how every chunk so far has been verified) with a testbench that checks its own results automatically and reports a single, unambiguous PASS or FAIL verdict per instruction, plus a final summary count. This is the style of testbench expected in any real verification role, and it is also simply less error-prone for a demo or viva, where a wall of raw numbers is hard for an audience (or a tired student) to check by eye.

14.2 Design of the Checking Task

A single task, `check`, applies one instruction and then compares ten of its outputs (the instruction type, the immediate, and all eight signals from the control unit) against expected values passed in as arguments. `rd`, `rs1` and `rs2` are not re-checked here because Chunks 2 and 6 already verified field extraction directly; Chunk 7 focuses on the two things a downstream processor would actually act on – the immediate value and the control decisions.

14.3 File: `tb/tb_selfcheck.sv`

```
`timescale 1ns/1ps
import riscv_pkg::*;
module tb_selfcheck;
    logic [31:0]  instr;
    logic [6:0]   opcode, funct7;
    logic [4:0]   rd, rs1, rs2;
    logic [2:0]   funct3;
    logic [31:0]  imm;
    instr_type_e  instr_type;
    logic reg_write, mem_read, mem_write, mem_to_reg, alu_src, branch, jump;
    logic [3:0] alu_ctrl;

    int pass_count = 0;
    int fail_count = 0;

    riscv_instr_decoder dut (.*);

    task check(input [31:0] i, input string name, input instr_type_e exp_type,
               input logic signed [31:0] exp_imm, input logic exp_rw, input logic exp_mr,
               input logic exp_mw, input logic exp_m2r, input logic exp_asrc,
               input logic exp_br, input logic exp_jmp, input logic [3:0] exp_alu);
        logic ok;
        instr = i;
        #10;
        ok = (instr_type == exp_type) && (imm === exp_imm) && (reg_write === exp_rw) &&
             (mem_read === exp_mr) && (mem_write === exp_mw) && (mem_to_reg === exp_m2r) &&
             (alu_src === exp_asrc) && (branch === exp_br) && (jump === exp_jmp) &&
             (alu_ctrl === exp_alu);
        if (ok) begin
            pass_count++;
            $display("PASS  %-6s type=%-7s imm=%0d", name, instr_type.name(), $signed(imm));
        end else begin
            fail_count++;
            $display("FAIL  %-6s type=%-7s imm=%0d",
                     name, instr_type.name(), $signed(imm));
            $display("      rw=%0d mr=%0d mw=%0d m2r=%0d asrc=%0d br=%0d jmp=%0d alu=%0d",
                     reg_write, mem_read, mem_write, mem_to_reg, alu_src, branch, jump, alu_ctrl);
        end
    endtask
```

Figure 14.1 – tb/tb_selfcheck.sv, part 1 of 3: signal declarations and the `check` task. `===` (rather than `==`) is used so that an unknown (`x`) value is treated as a real mismatch instead of silently comparing as false.

```
    initial begin
        $dumpfile("tb_selfcheck.vcd");
        $dumpvars(0, tb_selfcheck);

        //         instr         name    type     imm   rw mr mw m2r asrc br jmp alu
        check(32'h003100B3, "ADD",  R_TYPE,    0, 1, 0, 0, 0, 0, 0, 0, ALU_ADD);
        check(32'h40628233, "SUB",  R_TYPE,    0, 1, 0, 0, 0, 0, 0, 0, ALU_SUB);
        check(32'h009473B3, "AND",  R_TYPE,    0, 1, 0, 0, 0, 0, 0, 0, ALU_AND);
        check(32'h00C5E533, "OR",   R_TYPE,    0, 1, 0, 0, 0, 0, 0, 0, ALU_OR);
        check(32'h00F746B3, "XOR",  R_TYPE,    0, 1, 0, 0, 0, 0, 0, 0, ALU_XOR);
        check(32'h0128A833, "SLT",  R_TYPE,    0, 1, 0, 0, 0, 0, 0, 0, ALU_SLT);
        check(32'h00A30293, "ADDI", I_TYPE,   10, 1, 0, 0, 0, 1, 0, 0, ALU_ADD);
        check(32'hFFB30293, "ADDI2",I_TYPE,   -5, 1, 0, 0, 0, 1, 0, 0, ALU_ADD);
        check(32'h00F1F113, "ANDI", I_TYPE,   15, 1, 0, 0, 0, 1, 0, 0, ALU_AND);
        check(32'h0081E113, "ORI",  I_TYPE,    8, 1, 0, 0, 0, 1, 0, 0, ALU_OR);
        check(32'h00812083, "LW",   I_TYPE,    8, 1, 1, 0, 1, 1, 0, 0, ALU_ADD);
        check(32'h00322623, "SW",   S_TYPE,   12, 0, 0, 1, 0, 1, 0, 0, ALU_ADD);
        check(32'h00208463, "BEQ",  B_TYPE,    8, 0, 0, 0, 0, 0, 1, 0, ALU_SUB);
        check(32'hFE209EE3, "BNE",  B_TYPE,   -4, 0, 0, 0, 0, 0, 1, 0, ALU_SUB);
        check(32'h123452B7, "LUI",  U_TYPE, 32'h12345000, 1, 0, 0, 0, 1, 0, 0, ALU_ADD);
        check(32'h064000EF, "JAL",  J_TYPE,  100, 1, 0, 0, 0, 0, 0, 1, ALU_ADD);
```

Figure 14.2 – tb/tb_selfcheck.sv, part 2 of 3: sixteen checks (all fifteen instructions from Table 3.1, plus a second ADDI with a negative immediate to specifically exercise sign extension one more time at the fully-integrated level).

```
        $display("----------------------------------------");
        if (fail_count == 0)
            $display("ALL %0d TESTS PASSED", pass_count);
        else begin
            $display("%0d PASSED, %0d FAILED", pass_count, fail_count);
            $error("Self-check testbench reported failing tests - see FAIL lines above.");
        end

        $finish;
    end
endmodule
```

Figure 14.3 – tb/tb_selfcheck.sv, part 3 of 3: the final summary and pass/fail verdict.

Compile and run with:

```
iverilog -g2012 -o sc.vvp rtl/riscv_pkg.sv rtl/field_extractor.sv rtl/imm_generator.sv ^
       rtl/instr_type_decoder.sv rtl/control_unit.sv rtl/riscv_instr_decoder.sv tb/tb_selfcheck.sv
vvp sc.vvp
```

Expected output – sixteen PASS lines followed by a clean summary:

```
PASS  ADD    type=R_TYPE imm=0
PASS  SUB    type=R_TYPE imm=0
PASS  AND    type=R_TYPE imm=0
PASS  OR     type=R_TYPE imm=0
PASS  XOR    type=R_TYPE imm=0
PASS  SLT    type=R_TYPE imm=0
PASS  ADDI   type=I_TYPE imm=10
PASS  ADDI2  type=I_TYPE imm=-5
PASS  ANDI   type=I_TYPE imm=15
PASS  ORI    type=I_TYPE imm=8
PASS  LW     type=I_TYPE imm=8
PASS  SW     type=S_TYPE imm=12
PASS  BEQ    type=B_TYPE imm=8
PASS  BNE    type=B_TYPE imm=-4
PASS  LUI    type=U_TYPE imm=305418240
PASS  JAL    type=J_TYPE imm=100
----------------------------------------
ALL 16 TESTS PASSED
```

NOTE: If even one line says FAIL, the extra fields printed on that line (`rw`, `mr`, `mw`, and so on) show the actual simulated values, which can then be compared field by field against Table 12.1 to pinpoint exactly which signal is wrong – far faster than re-deriving the expected values from scratch.

14.4 Waveform Verification in GTKWave

A passing self-checking testbench proves correctness numerically, but being able to also show and explain a waveform is expected in a digital design viva, and is genuinely useful for building intuition about how the design behaves over time.

1. Run the simulation exactly as in Section 14.3 – this produces `tb_selfcheck.vcd` in the current folder.
2. Open it with `gtkwave tb_selfcheck.vcd`.
3. In the left “SST” panel, click on `tb_selfcheck`, then on `dut` underneath it, to reach the signals of the top-level decoder itself rather than the testbench wrapper.
4. Select `instr`, `instr_type`, `imm`, `reg_write`, `alu_ctrl`, `branch` and `jump`, and click “Append” (or drag them) into the waveform pane on the right.

5 Click the “zoom to fit” button (usually shown as a magnifying glass with an outward arrow) so the entire 160 ns simulation (sixteen instructions, 10 ns apart) is visible in one view.

6 Right-click `instr` and `imm` in the waveform pane and set their data format to Hex (for `instr`) and Signed Decimal (for `imm`) from the right-click menu, so the values are directly readable instead of showing as raw binary.

With this view open, `instr_type` can be watched changing from `R_TYPE` across the first six instructions, to `I_TYPE` for the next five, then `S_TYPE`, `B_TYPE` (twice), `U_TYPE`, and finally `J_TYPE` – visually confirming the same classification already checked numerically in Figure 14.2, and a good one-screenshot summary to include in a project report or slide.

> **NOTE:** To keep this exact view for later without repeating the click-and-drag steps, GTKWave can save a session with File > Write Save File, producing a `.gtkw` file. Reopening with `gtkwave tb_selfcheck.vcd tb_selfcheck.gtkw` restores the same signal layout instantly – useful to commit to the repository so both team members see an identical waveform view.

14.5 Definition of Done

- [ ] `tb_selfcheck.sv` prints "ALL 16 TESTS PASSED" with zero FAIL lines.
- [ ] The waveform has been opened in GTKWave at least once, with instr, instr_type, imm and the control signals visually inspected end to end.
- [ ] Both team members can point at any single instruction in the waveform and correctly state, out loud, its type, its immediate, and whether it writes a register, reads memory, writes memory, or branches/jumps – this is the single best rehearsal for a viva question.
- [ ] Changes committed and pushed to `working`, then merged into `main` once Abhimanyu confirms the output.

15 Chunk 8 – Documentation and Submission Readiness

CHUNK OWNER : ABHIMANYU

15.1 Objective

Turn a working pile of files into a project that a third party – an evaluator, a professor, or future the team members themselves – can pick up, understand, and re-run without needing either team member to explain it in person. This chunk produces no new RTL logic; it consolidates and polishes everything already built.

15.2 Final `README.md`

Replace the Chunk 1 starter README with a complete one. At minimum it should contain the sections below; the content for each has already been produced somewhere earlier in this guide and mostly needs to be assembled rather than freshly written.

README section	Source material already available
Project description	Section 1.1 and 1.3
Team members	Already in the Chunk 1 starter README
Supported instructions	Table 3.1
Repository layout	Section 5.1
Tools required	Section 4.1
How to build and simulate	The `iverilog` / `vvp` commands from Section 13.5 and 14.3
How to view waveforms	Section 14.4
Known limitations / future work	Section 3.3 and Section 18

15.3 Code Cleanup Pass

Working through every file under `rtl/` and `tb/` one more time, in order:

- Confirm every module has a one-line comment above it stating what it does (most already do, from the figures in this guide).
- Remove any leftover `$display` debug lines that were added informally while chasing a bug and were never part of the figures in this guide.
- Confirm indentation is consistent (4 spaces, as used throughout every code figure in this guide) and that no tabs have been mixed in – VS Code's status bar shows “Spaces: 4” or “Tab Size: 4” in the bottom right; click it to convert a file if it shows otherwise.
- Delete any stray `.vvp` or `.vcd` files that may have been accidentally committed before the `.gitignore` from Chunk 1 existed (`git rm --cached` removes a file from Git's tracking without deleting it from disk, if needed).

15.4 Final Full Regression

Run every testbench built across all eight chunks, one after another, and confirm every single one still passes exactly as it did when its own chunk was completed. This catches the one mistake most likely to happen this late in a project: an edit made during cleanup that accidentally breaks a module that was already working.

```
iverilog -g2012 -o t1.vvp tb/sanity_and_gate.sv tb/sanity_and_gate_tb.sv && vvp t1.vvp

iverilog -g2012 -o t2.vvp rtl/field_extractor.sv tb/tb_field_extractor.sv && vvp t2.vvp

iverilog -g2012 -o t3.vvp rtl/riscv_pkg.sv rtl/imm_generator.sv ^
       tb/tb_imm_generator.sv && vvp t3.vvp

iverilog -g2012 -o t4.vvp rtl/riscv_pkg.sv rtl/instr_type_decoder.sv ^
       tb/tb_instr_type_decoder.sv && vvp t4.vvp

iverilog -g2012 -o t5.vvp rtl/riscv_pkg.sv rtl/control_unit.sv ^
       tb/tb_control_unit.sv && vvp t5.vvp

iverilog -g2012 -o t6.vvp rtl/riscv_pkg.sv rtl/field_extractor.sv rtl/imm_generator.sv ^
       rtl/instr_type_decoder.sv rtl/control_unit.sv rtl/riscv_instr_decoder.sv ^
       tb/tb_selfcheck.sv && vvp t6.vvp
```

NOTE: The `&&` between commands means “only run the next command if the previous one succeeded”, so a compile error in an earlier test stops the sequence instead of hiding among later output. Running Chunk 6's own testbench again here is optional since Chunk 7's self-check testbench already exercises the same top-level module more thoroughly.

15.5 Final Git Steps

1. Follow the standard cycle from Section 5.3 one last time for the README and cleanup changes.
2. Once merged into `main`, both team members should independently do a completely fresh clone of the repository into a new, empty folder (`git clone fresh-copy`) and repeat the full regression in Section 15.4 there. This is the single best way to catch a “works on my machine only” problem – for example a file that was created but never actually committed.
3. Optionally tag this final state for easy reference: `git tag v1.0 -m "Final submission"` followed by `git push origin v1.0`.

15.6 Definition of Done

- [ ] README.md contains every section listed in Table 15.1's left column, fully written out.
- [ ] Every command in Section 15.4 succeeds with no errors and prints the same PASS/expected output already established in its own chunk.
- [ ] A completely fresh clone of the repository, on a machine with the toolchain from Section 4 installed, builds and passes every test with no manual fixes.
- [ ] Both `working` and `main` branches are in sync (no unmerged commits sitting on `working`).

16 Reference Tables

16.1 Complete Opcode Map (This Project's Scope)

Opcode (binary)	Format	Instructions using it
`0110011`	R	ADD, SUB, AND, OR, XOR, SLT
`0010011`	I	ADDI, ANDI, ORI
`0000011`	I	LW
`0100011`	S	SW
`1100011`	B	BEQ, BNE
`0110111`	U	LUI
`1101111`	J	JAL

16.2 Complete Control Signal Table

This is the same information as Table 12.1, reproduced here for quick reference without needing to return to Chunk 5.

Instr	RegWr	MemRd	MemWr	M2Reg	ALUSrc	Branch	Jump	ALUCtrl
ADD	1	0	0	0	0	0	0	ADD
SUB	1	0	0	0	0	0	0	SUB
AND	1	0	0	0	0	0	0	AND
OR	1	0	0	0	0	0	0	OR
XOR	1	0	0	0	0	0	0	XOR
SLT	1	0	0	0	0	0	0	SLT
ADDI	1	0	0	0	1	0	0	ADD
ANDI	1	0	0	0	1	0	0	AND
ORI	1	0	0	0	1	0	0	OR
LW	1	1	0	1	1	0	0	ADD
SW	0	0	1	0	1	0	0	ADD
BEQ	0	0	0	0	0	1	0	SUB
BNE	0	0	0	0	0	1	0	SUB
LUI	1	0	0	0	1	0	0	ADD
JAL	1	0	0	0	0	0	1	ADD*

16.3 Two's Complement Refresher

Every negative immediate in this project is represented in two's complement, the standard way digital hardware represents signed numbers. To negate a number in two's complement: invert every bit, then add 1. For example, +5 as an 8-bit number is `00000101`; inverting gives `11111010`; adding 1 gives `11111011`, which is -5. This is exactly why a negative immediate, when printed as a plain unsigned decimal by `%0d` in Section 10.5, appears as a very large positive number close to 4,294,967,296 (2 to the power 32) – it is the same 32 bits, just interpreted without the sign convention.

16.4 Glossary

Term	Meaning
ISA	Instruction Set Architecture – the agreed vocabulary of operations and their bit-pattern encodings (Section 2.1).
RV32I	The RISC-V base integer instruction set using 32-bit registers (Section 2.3).
Opcode	The bottom 7 bits of every instruction, identifying its broad category/format.
funct3 / funct7	Extra bits that, together with the opcode, pin down the exact operation within a format.
Immediate	A constant number encoded directly inside an instruction rather than read from a register.
Sign extension	Copying an immediate's sign bit into all the new, wider bit positions so a small signed number keeps the correct value at a larger bit width.
Combinational logic	A circuit whose output depends only on its current inputs, with no memory of the past – the entire decoder in this project is combinational.
Testbench	Simulation-only code written to drive inputs into a design and observe or check its outputs; never part of the real hardware.
VCD	Value Change Dump – the waveform file format produced by `$dumpvars` and opened by GTKWave.
RTL	Register-Transfer Level – the level of description used for the actual synthesizable design files, as opposed to testbenches.

17 Viva and Presentation Preparation

The questions below are the ones most likely to come up, based directly on the design decisions made in this guide. Each is paired with where in this document the full answer already lives, so revising for a viva is a matter of re-reading specific sections rather than starting from scratch.

Likely question	Where the answer is
Why did you choose RISC-V instead of x86 or ARM?	Section 2.2 and 2.3
Why is your decoder split into four sub-modules instead of one big module?	Section 3.1 and 13.2
Why do B-type and J-type store their immediate bits in a scrambled order?	Sections 2.5.4, 2.5.6 and 2.6
What is sign extension and why is it needed?	Section 2.6 and 16.3
Why do BEQ and BNE produce identical control signals?	Note in Section 12.3
What would you need to add to turn this into a full CPU?	Section 18
Why does your field extractor output meaningless values for some fields on some instructions?	Note in Section 9.3
How did you verify your design is correct?	Sections 9 through 14, especially the self-checking testbench in Section 14
Why is this a purely combinational design with no clock?	Section 1.1 and the note in Section 6.3
What instructions does your decoder NOT support, and why?	Section 3.3

17.1 A 60-Second Verbal Summary

If asked to describe the project in one breath: “We built a combinational instruction decoder for a fifteen-instruction subset of RV32I, RISC-V's 32-bit base integer instruction set, covering all six of its instruction formats. The design is split into four independently-tested SystemVerilog modules – a field extractor, an immediate generator, an instruction-type classifier, and a control unit – combined into one top-level decoder. Every module was verified in Icarus Verilog against hand-calculated expected values before being combined, and the final design passes a self-checking testbench covering every supported instruction, which we also verified visually in GTKWave.”

17.2 Suggested Slide / Report Structure

1. Title slide: project name, both names, course details.
2. Motivation: what a decoder does inside a CPU (Section 1.3) and why RISC-V (Section 2.2–2.3).
3. The six instruction formats, using the diagrams from Section 2.5.
4. Architecture diagram of the four sub-modules (Figure 13.1).
5. One slide per sub-module: its job, and one worked example (Sections 9–12).
6. Verification approach: self-checking testbench results and a GTKWave screenshot (Section 14).
7. Scope and future work (Sections 3.3 and 18).

8 Live demo, if time allows: run `vvp sc.vvp` from Section 14.3 live and show “ALL 16 TESTS PASSED” on screen.

18 Future Scope and Extensions

None of the following is required for this project, but each is a natural, well-scoped next step that reuses the existing structure rather than requiring a redesign – useful to mention if asked “what would you do next”.

- **Complete RV32I coverage.** Add SLTI, SLLI/SRLI/SRAI (shifts), AUIPC and JALR by extending the existing `case` statements in `control_unit.sv` and `imm_generator.sv` – no new instruction format or technique would be required, only more branches following the same pattern already established.
- **A real ALU.** Build a module that accepts `alu_ctrl` (already produced by this decoder) plus two operands, and actually performs the ADD/SUB/AND/OR/XOR/SLT operation, turning `alu_ctrl` from a “described decision” into an “executed decision”.
- **A register file.** A 32-by-32-bit array of registers, read using `rs1`/`rs2` and written using `rd` and `reg_write`, both of which this decoder already produces.
- **A branch comparator.** Using `funct3` (already available) together with the ALU's zero flag to correctly distinguish `BEQ` from `BNE`, resolving the note in Section 12.3.
- **A single-cycle datapath.** Wiring the decoder, ALU, register file, a small data memory, and an instruction memory together into a complete, executing single-cycle RISC-V CPU – a natural capstone that this project's decoder is already built to plug directly into.
- **FPGA deployment.** Once simulated, the same RTL (everything under `rtl/`) can be synthesised onto an FPGA development board using vendor tools (for example Xilinx Vivado or an open-source flow such as Yosys), with no changes needed to the decoder logic itself.

19 Troubleshooting Guide

Symptom	Likely cause and fix
`'iverilog' is not recognized as an internal or external command`	Icarus Verilog is not on the Windows PATH. Reinstall and tick the PATH option (Section 4.2), then fully restart VS Code.
`syntax error` pointing at `logic`, `always_comb`, or `import`	The `-g2012` flag was left off the `iverilog` command. Every compile command in this guide includes it – add it back.
Simulation runs but every printed field is `x`	A port was left unconnected in an instantiation (Section 13.3), or a signal was declared but never driven. Check every named port connection in the failing module's instantiation for a typo.
A signal in GTKWave shows a flat red line	Red in GTKWave means an unknown (`x`) value, which almost always means that signal is never actually driven – check for a missing `default:` branch in a `case` statement (Section 6.5) or a missing wire connection.
`multiple driver` or similar conflict error	The same signal is being driven by more than one `assign` or by both an `assign` and an `always_comb` block. Each signal must have exactly one source.
Test output is close but one or two fields are wrong	Re-derive the expected value by hand using the bit-slicing method in Section 2.7 rather than trusting memory; cross-check the specific hex instruction against Table 3.1 / the relevant chunk's table for a transcription mistake.
`git pull` reports a merge conflict	Open the conflicted file in VS Code, use the built-in merge editor to choose "Accept Incoming", "Accept Current", or manually combine the two versions, save, then `git add` the file and `git commit` to finish the merge (Section 5.3).
`git push` is rejected ("failed to push some refs")	The remote branch has commits that are not present locally. Run `git pull origin working` first (resolving any conflict as above), then push again.
A teammate's cloned copy fails a test that passes locally	A required file was likely never committed. Run `git status` to check for untracked files, and confirm with `git log --stat` that the relevant file appears in a prior commit.
GTKWave opens but shows no signals at all	The wrong hierarchy level is selected in the left-hand SST panel. Click down into `tb_selfcheck` and then `dut` (Section 14.4) rather than expecting signals at the very top level.

NOTE: When truly stuck, isolate the problem the same way the chunks were built: strip the failing test down to the single smallest module and single instruction that reproduces it, following exactly the pattern already used in every chunk of this guide, rather than debugging the fully integrated design at once.

This concludes the guide. Following the eight chunks in Sections 8 through 15, in order, alternating between Aryan and Abhimanyu and merging into `main` after each verified chunk, is sufficient to take this project from an empty repository to a fully working, fully tested, and fully documented Mini RISC-V Instruction Decoder.